

FwdLLM+: Accelerating Forward-Only FedLLM With Low-Rank Perturbations

Chen Peng[✉], Mengwei Xu[✉], Zhenyan Lu[✉], Wei Liu[✉], Shangguang Wang[✉], *Senior Member, IEEE*,
Nicholas D. Lane[✉], Qibo Sun[✉], and Dongqi Cai[✉], *Member, IEEE*

Abstract—Federated Learning (FL) facilitates privacy-preserving fine-tuning of Large Language Models (LLMs) for mobile applications, termed FedLLM. A vital challenge of FedLLM is the tension between LLM complexity and resource constraint of mobile devices. In response to this challenge, we first introduce FwdLLM (our conference version), an innovative FL framework designed to enhance the FedLLM efficiency. The key idea of FwdLLM is to employ backpropagation (BP)-free training methods, requiring devices only to execute memory-efficient “perturbed inference”. Enabled by mobile NPU acceleration and an expanded array of participating devices, FwdLLM delivers substantially better wall-clock efficiency than BP-based FedLLM. However, FwdLLM based on vanilla BP-free optimization theoretically requires more optimization steps to converge. In this work, we further enhance FwdLLM to FwdLLM+, which incorporates advanced zeroth-order optimization techniques and low-rank perturbation decomposition to reduce convergence steps. Finally, we conduct extensive experiments on 4 models (ranging from 110 M to 7B) and 8 more datasets, demonstrating that FwdLLM+ achieves up to $151\times$ faster training, a 93% memory reduction compared to vanilla BP-based FedLLM, and superior performance compared to FwdLLM, enabling efficient federated fine-tuning of billion-parameter LLMs on commodity mobile devices.

Index Terms—Federated learning, large language models, low-rank decomposition, mobile devices, zeroth-order optimization.

I. INTRODUCTION

LARGE Language Models (LLMs) such as GPTs and LLaMA have showcased an impressive ability to

Received 11 July 2025; revised 10 January 2026; accepted 6 March 2026. Date of publication 10 March 2026; date of current version 6 July 2026. Recommended for acceptance by A. Kaushik. (Corresponding authors: Qibo Sun; Dongqi Cai.)

Chen Peng, Mengwei Xu, Shangguang Wang, and Qibo Sun are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: pickppcc@bupt.edu.cn; mwx@bupt.edu.cn; sgwang@bupt.edu.cn; qbsun@bupt.edu.cn).

Zhenyan Lu is with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the Department of Computer Science and Technology, University of Cambridge, CB2 1TN Cambridge, U.K. (e-mail: luzhenyan@bupt.edu.cn).

Wei Liu is with Xiaomi Corporation, Beijing 100085, China (e-mail: liuwei40@xiaomi.com).

Nicholas D. Lane is with the Department of Computer Science and Technology, University of Cambridge, CB2 1TN Cambridge, U.K., and also with Flower Labs., 22085 Hamburg, Germany (e-mail: ndl32@cam.ac.uk).

Dongqi Cai was with the Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the Department of Computer Science and Technology, University of Cambridge, CB2 1TN Cambridge, U.K. He is now with the School of Intelligence Science and Technology, Nanjing University, Suzhou, Suzhou 215163, China (e-mail: cdq@nju.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TMC.2026.3672496>, provided by the authors.

Digital Object Identifier 10.1109/TMC.2026.3672496

handle generic machine learning tasks. As foundational models, pre-trained LLMs can be fine-tuned for various downstream tasks and have been applied across a broad range of mobile applications, including but not limited to question answering, personal assistance, and data retrieval [1], [2], [3], [4], [5]. Early efforts have been invested to adapt LLMs to mobile devices while maintaining data privacy during the fine-tuning process. Often, these efforts employ federated learning, an approach known as FedLLM [6], [7], [8], [9], [10], [11].

A salient feature of LLMs is their scalability: by incorporating more parameters, LLMs can continually evolve, achieving higher accuracy or even emergent abilities [12], [13], [14], [15]. Consequently, contemporary LLMs have grown enormously in size and are hard to be trained even on a GPU cluster [16], [17], not to mention mobile devices. Recent research of FedLLM [6], [7], [9], [10] primarily addresses the network issue between devices and cloud aggregator, yet the convergence is still lengthy and being impractical for developers. Through pilot experiments (Section II-B), we identify three key obstacles towards practical FedLLM.

- *Huge memory footprint*: The predominant on-device training algorithm [18], [19] necessitates extensive memory to store intermediate results such as activations and gradients. Although fine-tuning could omit most gradients with layers frozen, activations continue to demand considerable memory, often exceeding device capabilities. For example, 3.9 GB is required for RoBERTa-large. It results in extra I/O time to swap in/out data [20], [21] and makes the training task a highly likely victim of mobile OS’s low memory killer [22]; in either way, the FedLLM convergence is significantly slowed down.

- *Incompatible with mobile accelerators*: Mobile SoCs are often furnished with powerful, fast-evolving DNN accelerators (NPUs), e.g., Google Edge TPU and Qualcomm Hexagon that are up to $30\times$ faster than CPUs. Regrettably, on-device training is unsupported on nearly all mobile NPUs, since they are tailored for inference rather than training, and thus lack the requisite support for training-specific operations like SELECT_OPS [23] and dynamic gradient updating.

- *Limited device scalability*: In FL, only dozens of devices participate in training simultaneously, even when millions of IoT/smartphone devices are available. For instance, Google’s deployed FL system samples merely around 1% of training-ready devices per round [24], because even a small number of devices can saturate learning performance, meaning additional devices do not further expedite convergence.

This work leverages a crucial observation: all above issues can be somehow traced to the use of backpropagation-based (termed **BP**) training algorithm [25], [26] on devices (details in Section II-B). This prompts an essential question: *is it feasible to replace backpropagation with a more mobile-friendly training algorithm, thereby reinvigorating the FedLLM protocol?*

FwdLLM. training with “perturbed inferences”: Our prior system FwdLLM [27] is the first FedLLM framework that replaces backpropagation with a forward-only “perturbed inference” scheme, where each device slightly perturbs model weights and uses the resulting loss changes to construct an unbiased gradient estimator. This BP-free design relies only on forward passes, making on-device training much more memory-efficient and NPU-friendly; see Section III-B1 for the formal forward-gradient formulation.

Such BP-free training algorithms [28], [29], [30], [31], [32] have been researched by the ML community for decades but received very little attention. Relying on only inferences, the computation on devices is much more memory efficient and NPU-compatible; more devices also scale to faster convergence to allow more perturbed inferences simultaneously.

While the idea is intriguing, FwdLLM’s design still faces a crucial challenge: (i) *PEFT restriction*: it only updates LoRA-style adapter layers, which typically costs 1-3 % accuracy compared with full-parameter tuning; (ii) *Blind perturbations*: gradients are estimated with full-rank Gaussian noise, ignoring the empirically low-rank structure of LLM gradients and hence converging slowly; (iii) *Heavy perturbation demand*: to offset the high variance, each client must evaluate tens to hundreds of perturbations, so FwdLLM is efficient only when a large client pool is available. Section III-A details how FwdLLM+ eliminates these issues.

FwdLLM+ overcomes this limitation by shifting from parameter-efficient fine-tuning to full-parameter fine-tuning, employing several optimization strategies as detailed in Section III-B2. To stabilize BP-free training in a larger optimization space, we first incorporate central-difference derivation to constrain the search range. We then decompose the full-sized perturbation into two low-rank structured matrices, reflecting the low-rank nature of LLM gradients. This decomposition, applied in the full-parameter space, improves gradient estimation accuracy without incurring the computational cost of multiple perturbations. As a result, FwdLLM+ achieves better convergence accuracy, fewer convergence steps, and improved adaptability to downstream tasks.

We have implemented both FwdLLM and FwdLLM+, and evaluated it on 4 typical transformer-based models: BERT-base (0.1B) [33], RoBERTa-large (0.3B) [34], OPT (1.3B) [35] and LLaMA (7B) [17] and 8 classic NLP tasks (both discriminative and generative). The on-device training is profiled on Google Pixel 7 Pro and Jetson TX2. The results demonstrate our system’s impressive performance: compared to full-model fine-tuning, FwdLLM reduces the training time from 10.9–97.9 hours to 0.2–0.8 hours (up to $217.3\times$ reduction); compared to more competitive baselines enhanced by different PEFT methods, FwdLLM still delivers $2.0\times$ – $93.4\times$ speedup ($10.6\times$ on average). FwdLLM also reduces the memory footprint by up to $14.6\times$ and $11.5\times$ compared to them, respectively. Furthermore, across all

tested models and tasks, the optimized system FwdLLM+ consistently demonstrated superior performance relative to FwdLLM. On BERT-series models [33], [34], FwdLLM+ achieved an average accuracy uplift of 5.6%. Notably, for generative tasks evaluated on OPT architectures, FwdLLM+ gains rise to 19.5%. Through orchestration with quantization (INT4), for the first time our system enables fine-tuning a billion-parameter model like LLaMA over COTS smartphones within only 10 minutes.

Contributions: We make following contributions.

- We introduce the first BP-free federated learning framework FwdLLM that facilitates practical and efficient federated LLM fine-tuning with inference-level memory footprint.
- Built on top of FwdLLM, FwdLLM+ overcomes PEFT-induced accuracy loss and FwdLLM’s low-precision issues in sparse perturbation regimes via zeroth-order optimization with low-rank perturbation decomposition.
- Through extensive experiments, we demonstrate that our system yields substantial improvements over existing BP-based baselines. Moreover, our system serves to unify the pathway of on-device inference and training. Rather than viewing these as two separate research domains that necessitate distinct methods and optimizations, our system enables researchers in mobile AI systems and hardware to concentrate on optimizing on-device inference. This focus, in turn, leads to more efficient federated learning processes. The code of our system is available at: <https://github.com/UbiquitousLearning/FwdLLM>.

II. BACKGROUND AND MOTIVATIONS

A. Federated Fine-Tuning of LLMs

Large language model has been a revolutionary technique for its superior performance in serving generic, complex, and few-shot ML tasks. Training LLMs typically includes two crucial steps: (i) *pre-training* that endows the models with generic, rich knowledge of images/languages/etc, which requires large amounts of public training datasets and computing devices; (ii) *fine-tuning* that adapts the pre-trained models for various downstream tasks, which relies on domain-specific, privacy-sensitive data. Towards a privacy-friendly LLM training pipeline, there is a trend to combine LLM fine-tuning with federated learning, e.g., FedLLM [6], [7], [36].

We are at very early stage towards practical FedLLM. The gap between the tight resource constraint of edge devices and the extensive resource demand of on-device LLM training is huge, as demonstrated in both prior studies [6], [18], [19], [37] and the following experiments. Recent attempts incorporate parameter-efficient fine-tuning techniques (LoRA [38], adapters [39], and prompt tuning [40]) into FedLLM and see significant improvements in saving the network traffic between devices and aggregator. Yet they do not fully address many other issues such as excessive memory footprint.

B. Preliminary Experiments of FedLLM

- *FedLLM is hindered by the memory wall:* In this subsection, we reveal three crucial issues faced by FedLLM through pilot

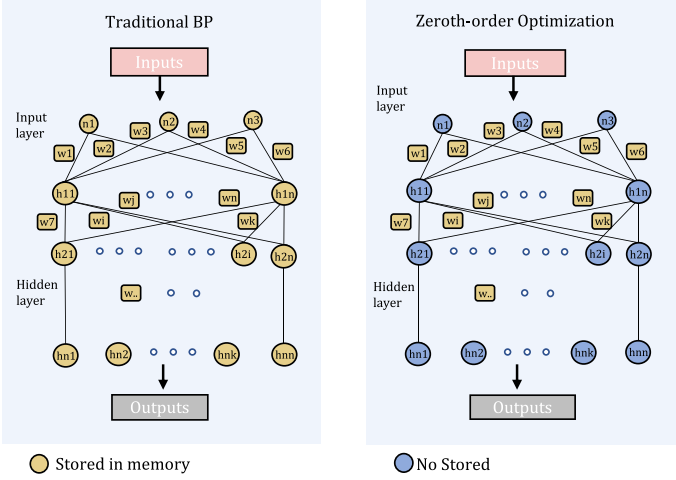


Fig. 1. A brief comparison of BP and ZO (Zeroth-order Optimization), we compare the standard backpropagation (BP) method with the FwdLLM approach. The BP method requires storing activations and gradients during the backward pass, whereas FwdLLM only requires perturbed inferences, which significantly reduces memory usage.

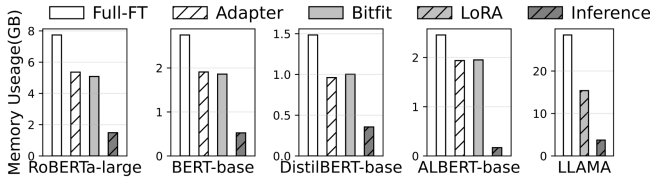


Fig. 2. Peak memory footprint of different training methods and inference. Batch size: 8.

experiments. And Fig. 2 shows the peak memory usage in training various LLMs with a relatively small batch size (8). The observed memory expense is often unaffordable for edge devices, e.g., more than 7.7 GBs for RoBERTa-large and 2.5 GBs for ALBERT-base, while the typical mobile devices possess only 4 GBs–12 GBs DRAM. More severely, on smart phones, even only a small portion of that memory could be used for training tasks to not compromise user experience [6], [41], [42]. In contrast, inference consumes much less memory (e.g., less than 1 GB) as it does not need to hold the intermediate computing results in memory as backpropagation does, which linearly scales up with batch size and sequence length.

PEFTs like adapter and bitfit methods cannot fundamentally reduce memory footprint as illustrated in Fig. 2. They bring only 21.2%–35.2% memory savings across different models, which is inadequate to fit certain large models like RoBERTa-large or LLaMA into real mobile devices. We then break down the memory consumption of RoBERTa-large and summarize the results in Table I. It explains why reducing trainable parameters cannot bring as significant memory saving: the activations generated during forward pass take up most of the memory usage, which cannot be eliminated even if the weights are not to be updated, as shown in Fig. 1.

• *FedLLM's inability to leverage powerful mobile accelerators*: Modern mobile devices frequently come equipped with high-end NN accelerators. As Moore's Law approaches its

TABLE I
THE BREAKDOWN OF MEMORY FOOTPRINT. MODEL: ROBERTA-LARGE; BATCH SIZE: 8. "FT": FINETUNING. "ACTIVATIONS" CONTAIN THE ACTIVATIONS FOR BACKWARD-GRADIENT COMPUTATION AND OPTIMIZER STATES.

Algorithms	Trainable Parameters	Memory Footprint (GB)			
		Weights	Activations	Gradients	Total
FT-full	354.3M (100%)	1.3	5.1	1.3	7.7
FT-adapter	3.2M (9.0%)	1.3	3.9	0.02	5.2
FT-bitfit	0.3M (0.8%)	1.3	3.8	0.009	5.1
FT-lora	0.8M (2.2%)	1.3	3.8	0.01	5.1
Inference	/	1.3	0.2	0	1.5

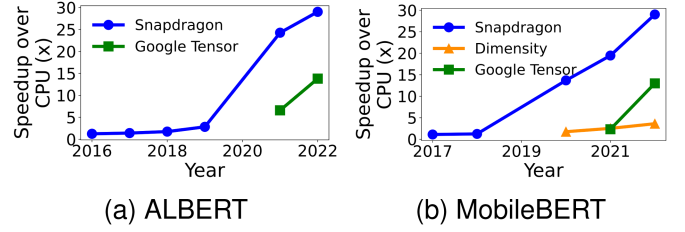


Fig. 3. Performance evolution of mobile NPU. Numbers are from AI Benchmark [43].

TABLE II
PER-BATCH (BS=8) TRAINING AND INFERENCE TIME ON GOOGLE PIXEL 7 PRO. LIBRARY: LLAMA.CPP FOR LLAMA AND TFLITE FOR OTHERS. "N/A": NOT SUPPORTED. * IS EMULATED IN AN INFINITE MEMORY ENVIRONMENT.

Model	Training Time (sec)			Inference Time (sec)		
	CPU	GPU	NPU	CPU	GPU	NPU
ALBERT	17.5	N/A		6.7	1.5	0.3
DistilBERT-base	6.9			3.4	0.8	0.2
BERT-base	14.0			6.9	0.8	0.3
RoBERTa-large	*28.1			11.7	2.9	0.8
LLaMA-7B (INT4)	N/A			22.1	N/A	N/A

limits, ASIC-based accelerators offer a promising pathway to sustain the growth in device capability in tandem with increasing model complexity. Fig. 3 summarizes the speedup achieved by NPUs over CPUs for three popular mobile chip series: Qualcomm Snapdragon (Hexagon), Google Pixels (Tensor TPU), and MediaTek (Dimensity). The findings are compelling: the NPU speedup rate is steadily rising and reaches nearly 30× on the recent Snapdragon 8+ Gen 1 chip. This underscores the imperative to utilize NPUs for efficient on-device DNN execution.

Nevertheless, on-device training can hardly benefit from mobile NPUs. In Table II, our measurements reveal the degree to which NPUs can accelerate DNN inference/training on the Google Pixel 7 Pro. While mobile NPUs significantly reduce inference latency compared to mobile CPUs and GPUs, they offer no support for DNN training. The reason behind this limitation is clear: these NPUs are tailored for inference tasks and thus lack the requisite support for backpropagation-specific operators such as BroadcastGradient, ReluGrad, StridedSliceGrad, and others [44]. Recent work [37] has enabled DNN training on Snapdragon Hexagon DSP, but this approach (i) compromises model accuracy due to lower data precision, and (ii) faces scalability challenges with other more proprietary NPUs, such as those found in Google Pixels and Huawei smart-phones.

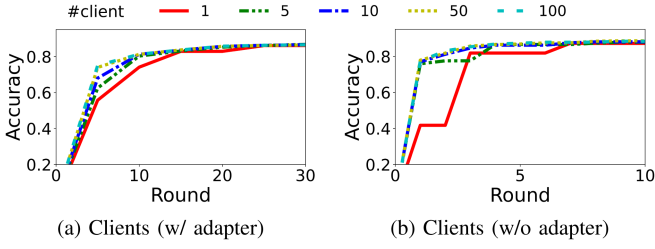


Fig. 4. Backpropagation-based FL has low device scalability.

TABLE III

PRIOR FL LITERATURE (MOBILE/SYSTEM/AI) USE A SMALL RATIO OF DEVICES IN EXPERIMENTS. MAXIMAL NUMBERS ARE SELECTED IF MANY DATASETS ARE USED.

Literature	Venue	Year	Total Devices	Devices per Round
Hermes [45]	MobiCom	2021	2,414	20 (0.8%)
PyramidFL [46]	MobiCom	2022	342,477	50 (0.01%)
FedAdapter [6]	MobiCom	2023	1,000	15 (0.15%)
FedBalancer [47]	MobiSys	2022	915	100 (10.9%)
Oort [48]	OSDI	2021	1,600,000	100 (0.006%)
FedNLP [49]	NAACL	2022	100	10 (10%)
C2A [50]	ACL	2023	100	25 (25%)
GradMA [51]	CVPR	2023	100	50 (50%)
FedScale [52]	ICML	2022	1,660,820	100 (0.006%)
Fjord [53]	NeurIPS	2021	3,400	10 (0.3%)

• *FedLLM has low device scalability*: Backpropagation-based FL struggles to enhance its convergence speed with more participant devices. As depicted in Fig. 4, involving merely tens of devices per round leads to the saturation of convergence speed, regardless of the utilization of PEFT techniques. Escalating the device count to 100 results in only marginal improvements; for example, it yields just a $1.04\times$ acceleration to reach an accuracy of 86% (with adapters).

In real-world scenarios, there can easily be millions of devices, such as smartphones and IoTs, that are capable of contributing local data and computing resources. After filtering out devices in non-optimal conditions, such as those with low battery or high utilization, the remaining training-available device number still easily exceeds hundreds. For instance, in Google’s deployed FL system [24], around 10,000 devices are simultaneously available for local training. The failure to scale to a larger number of idle devices seriously constrains the rate at which the model can converge. As indicated in Table III, even when millions of devices are available, the existing literature on FL uniformly adopts the default experimental setting of using no more than 100 devices. The root cause of this scalability issue can be attributed to backpropagation-based optimizers [12], [13], [14] and can hardly be addressed from a systems perspective.

III. METHODOLOGY

A. System Overview

FwdLLM+ is a cloud-device framework designed to enable practical federated fine-tuning of LLMs across mobile devices. The key innovation is to abandon backpropagation-based gradient descent and instead use “perturbed inference” that poses

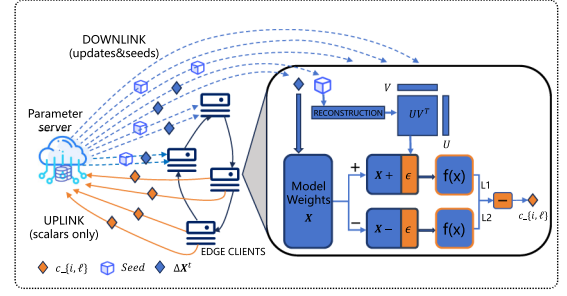


Fig. 5. FwdLLM+ workflow. At round t , the server broadcasts the aggregated update $\Delta\mathbf{X}^t$ (computed from round $t-1$) together with a fresh seed s^t that generates the low-rank factors (u_ℓ^t, v_ℓ^t) . Each client applies $\Delta\mathbf{X}^t$, runs two perturbed forward passes with $\pm\epsilon u_\ell^t v_\ell^{t-1\top}$, and computes a scalar directional derivative (coefficient) $c_{i,\ell}^t$ (shown as $c_{i,\ell}^t$ in the figure). Clients upload only these scalars for server-side aggregation.

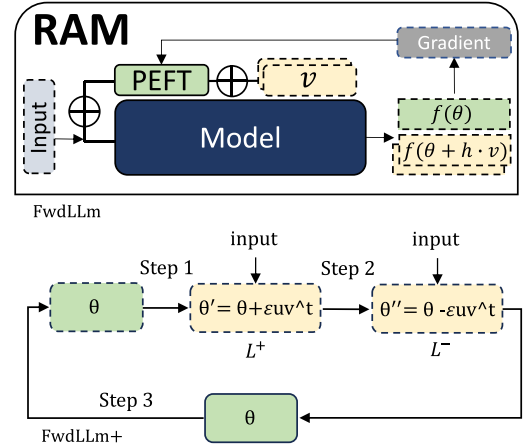


Fig. 6. Memory-Efficient Parameter Storage in FwdLLM and FwdLLM+. Dotted block will be released sequentially after computation. In FwdLLM+, $\theta = \text{size_of}(\mathcal{M})$. At Step 1, random perturbation vectors u (size = θ ’s first dimension) and v (size = θ ’s second dimension) perturb the base parameters to $\theta' = \theta + \epsilon uv^\top$ (seed = st). At Step 2, a perturbation $-2\epsilon uv^\top$ is applied to θ' to yield θ'' . Finally, Step 3 reverts θ'' to the original θ via $\theta = \theta'' + \epsilon uv^\top$. This reversible process allows FwdLLM+ to store only one copy of θ (size = θ), while FwdLLM requires θ -sized memory + 2-trainable_param.

much less memory/compute pressure on mobile devices and can be accelerated by energy-efficient mobile NPUs.

Simplified workflow illustration under OPT fine-tuning: As illustrated in Fig. 5, FwdLLM+ employs a parameter-server architecture similar to the traditional FedAvg protocol but fundamentally differs in its local computation strategy.

① *Downlink (server $\rightarrow$ clients)*: For $t = 0$, the server sends the full model $\mathbf{X}^0$ together with the seed s^0 , perturbation magnitude ϵ , and rank r . For $t \geq 1$, the server broadcasts two pieces of information: (i) the aggregated model update from round $t-1$, denoted as $\Delta\mathbf{X}^t = -\eta^{t-1} \sum_{\ell=1}^L \bar{c}_{\ell}^{t-1} u_{\ell}^{t-1} v_{\ell}^{t-1\top}$, where $\bar{c}_{\ell}^{t-1}$ is the server-side aggregate (e.g., average) of clients’ uploaded scalar directional derivatives $\{c_{i,\ell}^{t-1}\}$ for the direction $u_{\ell}^{t-1} v_{\ell}^{t-1\top}$. This update has the same shape as the full model parameters and can be applied directly by clients; and (ii) a fresh seed s^t to deterministically generate the low-rank factors (u_ℓ^t, v_ℓ^t) for computing new directional derivatives in round t .

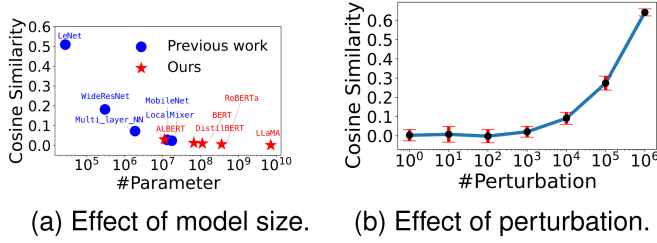


Fig. 7. Effectiveness of forward gradients. (a) Increased model parameters make the generated forward gradient unreliable. (b) Adding perturbations could make the forward gradient computed more similar to gold gradients.

② *Local update and uplink (clients $\rightarrow$ server)*: Upon reception ($t > 0$), each client first applies the received update $\Delta \mathbf{X}^t$ to obtain $\mathbf{X}_i^t$. Then, using the new seed s^t , the client perturbs the model by $\pm \epsilon u_\ell^t v_\ell^{t\top}$ and computes a scalar directional derivative $c_{i,\ell}^t = (f_i(\mathbf{X}^t + \epsilon u_\ell^t v_\ell^{t\top}) - f_i(\mathbf{X}^t - \epsilon u_\ell^t v_\ell^{t\top})) / (2\epsilon)$. The client uploads only $\{c_{i,\ell}^t\}$ (one scalar per layer/direction), making the uplink cost negligible; the downlink includes the explicit update $\Delta \mathbf{X}^t$ (model-sized) and the new seed s^t .

Consequently, only the first round ($t=0$) requires sending the full model; in subsequent rounds the server broadcasts the aggregated update $\Delta \mathbf{X}^t$ from the previous round together with a fresh seed s^t that defines the current perturbation directions, while clients upload only scalar directional derivatives. This process repeats until convergence.

FwdLLM+’s advantages by design. (1) *Computational Efficiency*: For each client, calculating a forward gradient is equivalent to executing inferences twice (i.e., $f(\mathcal{M}^+)$ and $f(\mathcal{M}^-)$), making its execution on NPU 19.5–27.5 $\times$ faster than computing one backward gradient on a CPU, as previously demonstrated in Table II. (2) *Memory Efficiency*: FwdLLM+ achieves significant memory savings compared to both standard backpropagation (BP) and original FwdLLM. Unlike BP which requires storing intermediate activations (typically dominating memory usage), as illustrated in Fig. 1, FwdLLM and FwdLLM+ avoids these entirely. Compared to original FwdLLM which stores $\text{size_of}(\mathcal{M}) + 2 \cdot \text{trainable_param}$, FwdLLM+ avoids the storage of perturbations (Fig. 6). Its peak memory footprint is approximately $\text{size_of}(\mathcal{M})$. Because it ingeniously employs seeds to deterministically generate perturbations on-the-fly, thereby eliminating the need to store large perturbation vectors in memory. (3) *Scalability and Precision with respect to client numbers*: As more clients compute forward gradients simultaneously, the aggregated gradient becomes closer to the real gradient, contributing to faster and more accurate convergence. Fig. 7(b) illustrates that the continuous addition of perturbations enables the forward gradients to estimate the true gradient with greater precision and stability.

Implications on privacy: From the cloud perspective, clients iteratively upload low-dimensional coefficients (scalar directional derivatives that reconstruct low-rank gradients server-side), i.e., approximated gradients that are unbiased estimations of the true gradients, similar to traditional FL methods. This approach is analogous to traditional backpropagation (BP)-based Federated Learning (FL) frameworks (e.g., FedPrompt [9]),

which are designed to be privacy-preserving. The fundamental privacy guarantee in such systems stems from training on decentralized data and ensuring that only model updates—be it parameters, prompts, or gradients—are aggregated by the central server, rather than users’ raw, private data. While this method prevents direct data exchange, naive parameter or gradient sharing can still be vulnerable to privacy leakage [54] (e.g., gradient inversion attacks).

Consequently, FwdLLM+ is designed to be compatible with the same advanced privacy techniques used to harden BP-based FL, such as differential privacy (DP) [55], secure aggregation [56], and homomorphic encryption [57]. In particular, regarding DP, FwdLLM+ is uniquely positioned to adopt the memory-efficient Scalar-wise DP mechanism proposed in recent ZO optimizations [55], [58]. Unlike standard BP-based DP which requires computationally expensive clipping of high-dimensional gradient norms, FwdLLM+ can strictly bound sensitivity by simply clipping the scalar directional derivatives and adding Gaussian noise before transmission. This ensures formal differential privacy guarantees without incurring the heavy memory overhead typical of private LLM training.

While these additional privacy-preserving methods are not implemented in the current version of FwdLLM+, they represent a clear direction for future enhancement. Nonetheless, the current FwdLLM+ design already provides significantly stronger privacy protection than any centralized approach that requires direct access to user data.

Unique challenges introduced by BP-free training: BP free methods remove the memory / compute overhead of back-propagation by replacing it with perturbed forward passes, but scaling this idea to federated fine tuning of large language models (LLMs) exposes several intertwined obstacles. (i) BP-free training is not a panacea for all models. Existing studies [28], [29], [59] primarily validate its utility for diminutive models (a few million bytes), which are 1–3 orders of magnitude smaller than standard LLMs that we target. As shown in Fig. 7, with the increase of parameter size, the generated forward gradients deviate significantly from the true gradients. Thus, to obtain reliable forward gradients, the required perturbations per batch increase exponentially with the parameter size. (ii) The conference prototype FwdLLM mitigates on-device memory pressure by updating only Parameter Efficient Fine Tuning (PEFT) blocks such as LoRA. However, restricting optimisation to this low rank subspace incurs a 1–3% accuracy drop compared to full tuning. (iii) Empirical observations indicate that LLM gradients during fine-tuning often exhibit an inherent low-rank structure [60], [61]. However, existing methods frequently rely on perturbations that are sampled from a Gaussian distribution, resulting in gradient estimates that are nearly full-rank [62], [63], [64], [65]. This disparity between the estimated full-rank gradients and the true low-rank gradients leads to inefficiencies in the estimation process and can result in suboptimal performance. (iv) To tame this high variance each client has to evaluate tens to hundreds of perturbations, inflating latency and energy consumption and limiting compatibility with resourceconstrained mobile devices. These challenges collectively motivate FwdLLM+, which abandons PEFT and blind perturbations in

favour of lowrankaware, fullparameter optimisation, delivering higher accuracy and faster convergence with dramatically fewer perturbations.

B. Low-Rank Perturbation-Based Federated LLM Fine-Tuning

In this section, we introduce an enhancement to FwdLLM by upgrading the zeroth order optimizer with low-rank zeroth order optimizer, resulting in a new variant termed FwdLLM+. This approach addresses key limitations of the original FwdLLM, replacing its forward gradient method with low-rank optimizer to leverage the low-rank structure of gradients in large language models. FwdLLM+ improves fine-tuning efficiency and performance in FL settings without relying on plug-in PEFT modules like LoRA. We structure this section into three parts: (1) a recap of the forward gradient method with PEFT and its limitations, (2) the integration of low-rank decomposition of perturbations in FwdLLM+, (3) their comparison.

B1) Forward Gradient With Parameter-Efficient BP-Free Learning: In the original FwdLLM, the forward gradient method is paired with PEFT techniques like LoRA to enable efficient fine-tuning in FL, it is based solely on the directional derivative, which can be computed both precisely and efficiently via the perturbed inference [28]. Formally, to compute the directional derivatives of deep learning functions, denoted as f , with respect to a perturbation vector v at a point θ , the following equation can be used [26], [66]:

$$\nabla_v f(\theta) = \lim_{h \rightarrow 0} \frac{f(\theta + h \cdot v) - f(\theta)}{h}, \quad (1)$$

where $v \in \mathcal{N}(0, 1)$ represents the weight perturbations, h is the perturbation magnitude and $\nabla_v f(\theta)$ is the directional derivative of f at the model weight point θ in the direction v [67]. In simpler terms, $\nabla_v f(\theta)$ signifies the slope of f in the direction of v . The direction of steepest ascent can be identified using the gradient $\nabla f(\theta)$. Nevertheless, determining the gradient $\nabla f(\theta)$ can be computationally demanding, as it necessitates both a forward pass and a backward pass [28], which we have empirically validated in Section 2. As an alternative, we leverage the forward gradient g_v , which is more cost-effective to compute:

$$g_v(\theta) = (\nabla f(\theta) \cdot v)v, \quad (2)$$

where $g_v(\theta)$ is established as an unbiased estimator of the gradient $\nabla f(\theta)$ [28].

B2) Low-Rank Perturbation Strategy in FwdLLM+: FwdLLM+ replaces the original forward-gradient estimator with a low-rank zeroth-order (ZO) optimizer [68], inspired by LoRA's low-rank insights but applied directly to full model parameters without PEFT. Intuitively, building on LoRA's empirical success, we extend the low-rank principle to the zeroth-order regime: by constraining perturbations to low-rank factors, we enable efficient parameter fine-tuning with compact low-rank matrices, focusing updates on the most influential directions and improving practical convergence.

Formally, for each layer ℓ with trainable parameters X_ℓ , FwdLLM+ uses low-rank matrix perturbations $U_\ell V_\ell^\top$ to estimate the gradient. Intuitively, this is a structured "push" to parameters, and the resulting two-point loss difference measures a 1D

slope along the shared low-rank direction. As summarized in the workflow (III A, items ①–③), the server samples seeds that deterministically generate low-rank factors $U_\ell \in \mathbb{R}^{m_\ell \times r_\ell}$ and $V_\ell \in \mathbb{R}^{n_\ell \times r_\ell}$, with $r_\ell \ll \min\{m_\ell, n_\ell\}$, and broadcasts only these seeds to all clients (so clients only upload scalar coefficients later). Each client i then forms perturbed models $\{X_\ell \pm \epsilon U_\ell V_\ell^\top\}_{\ell=1}^L$ and evaluates its local loss $f_i(\cdot) := F(\cdot; \xi_i)$ yielding the two-point finite-difference term

$$\hat{\nabla}_{X_\ell} F(\mathbf{X}) = \frac{1}{K} \sum_{i=1}^K \frac{1}{2\epsilon} \left[F(\{X_\ell + \epsilon U_\ell V_\ell^\top\}_{\ell=1}^L; \xi_i) - F(\{X_\ell - \epsilon U_\ell V_\ell^\top\}_{\ell=1}^L; \xi_i) \right] \cdot \frac{U_\ell V_\ell^\top}{r_\ell}. \quad (3)$$

Here X_ℓ denotes the parameters of the ℓ -th layer, ϵ is the perturbation magnitude, and K is the number of participating clients in this round.

Equivalently, as in the workflow step ③, the client-side computation can be phrased in terms of a *scalar* directional derivative per layer. For each client i and each layer ℓ , we define

$$c_{i,\ell} := \frac{F(\mathbf{X} + \epsilon \mathbf{U} \mathbf{V}^\top; \xi_i) - F(\mathbf{X} - \epsilon \mathbf{U} \mathbf{V}^\top; \xi_i)}{2\epsilon}, \quad (4)$$

which is a scalar coefficient along the global low-rank direction $\mathbf{U} \mathbf{V}^\top$ (block-diagonal over layers). Each client uploads only its local coefficients $\{c_{i,\ell}\}_{\ell=1}^L$ (one scalar per sampled direction/layer), and the server aggregates them into

$$\bar{c}_\ell = \frac{1}{K} \sum_{i=1}^K c_{i,\ell}$$

for each layer ℓ .

As described in workflow step ④, these aggregated scalars, together with the shared seeds that regenerate (U_ℓ, V_ℓ) , are then used on the client side to reconstruct the low-rank gradient estimator

$$\hat{\mathbf{G}}^t = \sum_{\ell=1}^L \bar{c}_\ell (U_\ell V_\ell^\top / r_\ell),$$

and each client applies a full-parameter low-rank update $\mathbf{X}_i^{t+1} \leftarrow \mathbf{X}_i^t - \eta^t \hat{\mathbf{G}}^t$.

To enhance stability and exploration, a lazy sampling strategy is employed: Rather than resampling V_ℓ at every iteration, the server regenerates V_ℓ only every ν iterations (e.g., $\nu = 50$). This strategy maintains a fixed perturbation subspace spanned by V_ℓ over ν consecutive updates, which reduces abrupt parameter shifts while enabling sufficient exploration of low-dimensional subspaces. The interval ν serves as a critical hyperparameter that balances exploration efficiency against convergence stability.

B3) Comparison of FwdLLM and FwdLLM+: To elucidate the distinctions between the original FwdLLM and the enhanced FwdLLM+, we refer to Fig. 8, which contrasts their perturbation and update mechanisms for fine-tuning large language models (LLMs) in a federated learning (FL) setting. The figure comprises two diagrams: the upper illustrates FwdLLM, perturbing PEFT adaptation parameters, while the lower showcases

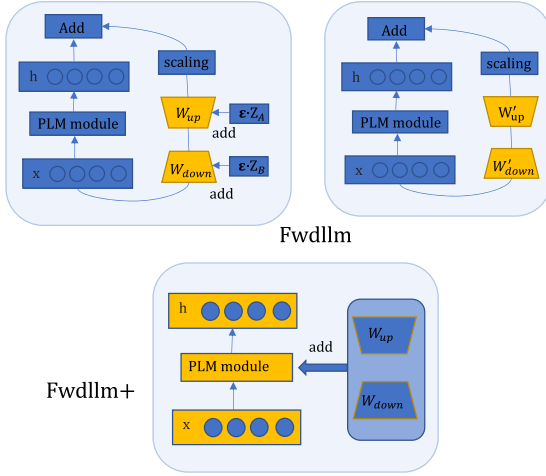


Fig. 8. Comparison of perturbation and update mechanisms in FwdLLM and FwdLLM+. The upper diagram illustrates the FwdLLM approach, where perturbations are applied to PEFT adaptation parameters (e.g., LoRA matrices W_{up} and W_{down}), while the lower diagram depicts the FwdLLM+ method, applying low-rank perturbations ϵUV^T directly to the trainable model parameters.

FwdLLM+, applying low-rank perturbations to the trainable model parameters. The key differences are outlined below:

- *Perturbation Mechanism:* In FwdLLM, perturbations are applied to PEFT parameters (e.g., LoRA matrices W_{up} and W_{down}), which are then added to the pre-trained language model (PLM) weights. Conversely, FwdLLM+ employs low-rank perturbations ϵUV^T directly on the trainable model parameters, bypassing PEFT constraints.
- *Parameter Scope:* FwdLLM fine-tunes only PEFT parameters, a small subset of the model, reducing memory demands but limiting expressiveness. FwdLLM+ targets the entire model parameters, enabling more comprehensive and accurate updates aligned with the low-rank nature of LLM gradients.

These distinctions highlight how FwdLLM+ overcomes the original FwdLLM's limitations, eliminating PEFT dependencies, boosting accuracy through full-parameter updates, and enhancing computational efficiency with low-rank perturbations, thus offering a superior approach for federated LLM fine-tuning. Moreover, because FwdLLM+ dispenses with PEFT-specific hyper-parameter searches (e.g., selecting and tuning LoRA modules/ranks) and the variance-threshold pacing hyperparameter used in FwdLLM, it can be readily scaled to a wider spectrum of tasks, models, and datasets with minimal tuning effort. This simplicity, together with the lower-variance gradient estimator analysed in our convergence study, also translates into faster practical convergence.

Illustrating PEFT's Expressiveness Bottleneck: When the preset LoRA rank is small, FwdLLM suffers from limited expressiveness, whereas FwdLLM+, operating on full-parameter perturbations, completely avoids this issue. LoRA approximates $\Delta W = BA$ where $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{d \times r}$ with $r \ll d$. The key limitation is that ΔW is constrained to the subspace $\text{col}(B) \otimes \text{row}(A)$. For example, consider learning $\Delta W = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$ with rank-1 LoRA. We need $\Delta W = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} [a_1, a_2] =$

TABLE IV

TESTED MODELS AND ON-DEVICE INFERENCE LIBRARIES. FOR FwdLLM AND PEFT BASELINES, WE USE THE LISTED PEFT CONFIGURATION.

Models	Arch.	Params.	PEFT	Infer. Libs
BERT-base [33]	Encoder-only	110M	Bitfit	TFLite [71]
RoBERTa-large [34]	Encoder-only	340M	Bitfit	TFLite [71]
LLaMA [17]	Decoder-only	7B	LoRA	llama.cpp [72]
Opt [35]	Decoder-only	1.3B	/	/

$\begin{bmatrix} b_1 a_1 & b_1 a_2 \\ b_2 a_1 & b_2 a_2 \end{bmatrix}$, which requires $b_1 a_1 = 1, b_1 a_2 = 2 \Rightarrow \frac{a_2}{a_1} = 2$ and $b_2 a_1 = 3, b_2 a_2 = 1 \Rightarrow \frac{a_2}{a_1} = \frac{1}{3}$. This is a contradiction here. While FwdLLM+ equals to conduct reinitialization every round: instead of $W_{\text{final}} = W_0 + \Delta W$, we do $W_1 = W_0 + B_1 A_1$, then reinitialize and compute $W_2 = W_1 + B_2 A_2$. The key insight is that each step solves a different optimization problem. The required update $W_{\text{target}} - W_1$ may be representable by new $B_2 A_2$ even if $W_{\text{target}} - W_0$ wasn't representable by $B_1 A_1$. By changing the starting point, we change the required direction.

IV. IMPLEMENTATION AND SETUPS

A. FwdLLM+ Prototype

We have fully implemented the FwdLLM+ prototype atop FedNLP [49], one of the state-of-the-art frameworks to develop and evaluate FL methods on NLP tasks. There are two primary ways to implement forward gradients: numerical differentiation and analytical differentiation. We use the former as it is almost identical to forward inference. We therefore implement it with functorch library [69]. For various PEFT methods: adapter is implemented with AdapterHub [39], a library that facilitates the integration of different pre-trained adapters for downstream tasks; LoRA and BitFit are implemented by ourselves. The quantized LLaMA training is based on AutoGPTQ library [70].

B. Models

We evaluate FwdLLM+ and FwdLLM on four popular LLMs, as shown in Table IV. Two of them are BERT-like models based on transformer encoders: (1) BERT-base (110 M) [33]; (2) RoBERTa-large (340 M) [34]. Those two models are extensively used in prior FedNLP research [6], [8], [49]. Apart from that, we also evaluate FwdLLM+ on the (3) OPT-1.3B (FP16) [35]. OPT belongs to the same family of decoder-only models like GPT-3. And on the SoTA open-sourced generative language model (4) LLaMA-7B (INT4) [17]. To hold the whole LLaMA model in memory, we quantize it to INT4 format using GPTQ [73]. FwdLLM (and the PEFT baselines) select different PEFT methods for different models using an offline profiler presented in Section III-B: LoRA [38] for LLaMA; and BitFit [74] for others. As far as we know, FwdLLM+ is the first attempt to apply federated learning to a billion-size models. The pre-trained weights of above models are from Huggingface [75].

C. Datasets

We experiment with two popular NLP datasets: (1) AG-NEWS [76] is a news classification dataset with 4 classes. The number of training samples for each class is 30 K and testing

1.9 K. (2) SQuAD-v1.1 (SQUAD) [77] is a commonly used version of stanford question answering dataset. By default, we uniformly divide the datasets into 10 clients for Squad and 1,000 clients for AGNEWS, and sample a subset of clients per round (default 100). Throughout all experiments we adopt the *time-to-accuracy* metric—measured until each dataset reaches its target accuracy (0.88 for AGNEWS)—and additionally log on-device runtime cost (peak memory, network traffic and energy consumption).

To enrich the evaluation and, more importantly, to benchmark the raw algorithmic efficiency of FwdLLM versus its enhanced variant FwdLLM+, we further select several representative tasks from SuperGLUE [78]: SST2 [79], RTE [80], WSC [81], CB [82] and WIC [83] for classification, together with COPA [84] for multiple-choice reasoning. These tasks collectively cover sentiment analysis, textual entailment and commonsense reasoning. To isolate the pure algorithmic difference between the two systems, we deliberately set the number of participating clients to just **one** for all SuperGLUE experiments, so that no advantage can be gained from client-side parallelism.

D. Hardware

As prior FL literature [45], [46], [48], [49], our experiments are carried out in a semi-emulation manner on two GPU servers each with $8 \times$ NVIDIA A100. The on-device training time is obtained on two popular edge devices: (1) Google Pixel 7 Pro (Pixel) is a popular mobile phone that is equipped with a Google Tensor G2 TPU, a Mali-G710 GPU and an Octa-core CPU. It runs Android 13 OS. We use TFLite [71] to train the four BERT-variant models and llama.cpp [72] to run inference with LLaMA. The inference and training speed is previously illustrated in Table II. (2) Jetson TX2 (TX2) [85] is a widely used edge board equipped with a 256-core NVIDIA Pascal GPU and a Dual-Core NVIDIA Denver 2 64-Bit CPU. We use PyTorch [86] for on-device inference and training. Note that Jetson TX2 has no NPU and is not the primary target platform of our system. Only our methods are supported by current mobile NPUs, whereas others can only be executed on CPUs.

E. Baselines

We compare FwdLLM and FwdLLM+ to the following baselines: (1) Full-FT always fine-tunes the whole model [49]. (2) Adapter tuning (Adapter) introduces a small tunable module between transformer layers and freeze other parameters [39], [87]. (3) Bias tuning (BitFit) only tunes bias of each layer in the LLM [74]. (4) FedAdapter is the SoTA FedNLP fine-tuning framework that incorporates adapter tuning with layer freezing techniques [49], along with a progressive training paradigm to identify the optimal adapter configuration automatically [6]. (5) LoRA tuning (LoRA) fine-tunes low-rank adapter matrices while freezing the remaining parameters, serving as a widely used PEFT baseline [38]. (6) MeZO (MeZO) is a memory-efficient zeroth-order optimizer that estimates gradients from forward passes only [62]. Except MeZO, all baselines use BP-based training; thus, they are limited to using smartphone CPUs, as previously discussed in Section II-B. In contrast, FwdLLM+

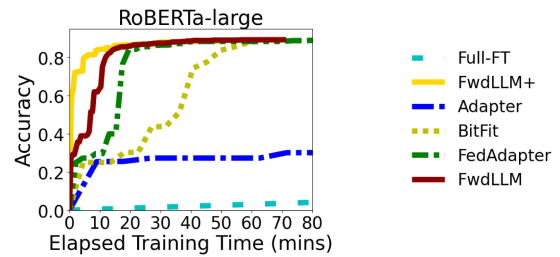


Fig. 9. Performance of our methods and baselines on AGNEWS dataset.

and FwdLLM can also utilize mobile GPUs and NPUs. A recent work [37] ports training to mobile NPU, yet does not support transformer models.

F. FL Settings

Unless otherwise stated, we employ **100** clients per round in all experiments, aligned with prior work [6], [45], [49], [53]. The only exception is the direct comparison between FwdLLM and its enhanced version FwdLLM+ on the SuperGLUE tasks, where we deliberately set the number of participating clients to **1** to isolate algorithmic efficiency from client-side parallelism. FwdLLM+ and all baselines use the same set of hyper-parameters as prior work [6], [49] in AGNEWS, SQUAD datasets: local training epoch as 1; mini-batch size as 8; learning rate as 0.01; max sequence length as 64 for AGNEWS and 256 for others. For datasets in SuperGLUE, we use the following set: batch size as 16, learning rate as $1e-7$. The default FL aggregating algorithm is FedSGD [88]. Network bandwidth is set to 10 Mbps by default as prior literature [6], [8], [49]. We naturally support heterogeneous/low-end clients since FwdLLM+ only requires lightweight on-device forward computations (without local backpropagation), reducing client-side compute and peak memory. We follow standard FL partial participation (and optional round deadlines/timeouts) so stragglers can skip a round without stalling aggregation.

V. EVALUATION

A. Convergence Performance on Small Language Models

We first evaluate the performance of our system on language models ranging from 110 M to 1.3B parameters. We use encoder-only BERT series models (BERT and RoBERTa Large), which have fewer than 1B parameters and are widely adopted in prior work [6], [8], [8], [49]. To obtain more general results, we also assess the effectiveness of our system on the decoder-only OPT-1.3B model.

Our system achieves the fastest convergence on mobile NPU and delivers the highest final accuracy across the majority of datasets: Fig. 9 illustrates the convergence process under the default setting. In this end-to-end experiment, we increase the per-round client participation to 1000 to maximize parallelism and reach higher performance. To reach the target accuracy, FwdLLM+ is $1.14\times$ faster than FwdLLM. And FwdLLM outperforms Full-FT by a significant factor of $132.7\times$. Compared

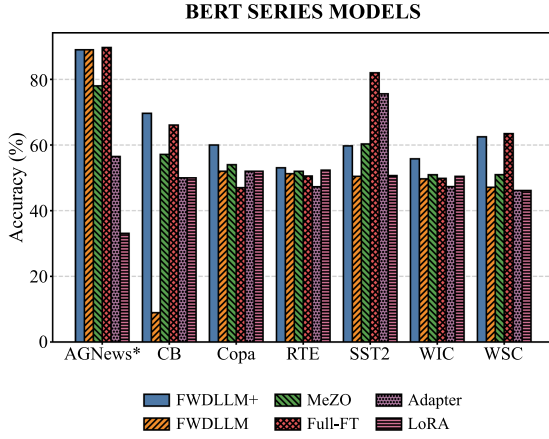


Fig. 10. Convergence performance of our system and baselines on BERT Series Models. Results marked with an asterisk (*) are obtained on RoBERTa-large, whereas those without an asterisk are obtained on BERT.

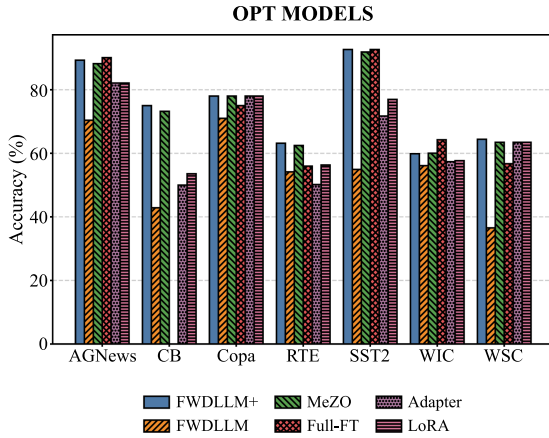


Fig. 11. Convergence performance of our system and baselines on OPT.

with parameter-efficient fine-tuning baselines and the state-of-the-art federated fine-tuning system FedAdapter [6], FwdLLM+ can still beat them non-trivially ($10.03\times$ on average), while FwdLLM also enjoys a sizeable speed-up of $9.6\times$ on average. The significant boost in performance partly stems from our system’s capacity to harness the powerful NPU, accelerating the training process. Unique to our system is its dependency solely on the forward pass, in contrast to other baselines that require the backward pass — a function not yet compatible with NPUs, as elaborated in Section II-B.

To further highlight the improvement that FwdLLM+ brings over its predecessor FwdLLM, we conduct an additional set of experiments on seven representative language-understanding datasets: six SuperGLUE sub-tasks (COPA, CB, RTE, WIC, SST2, and WSC) together with the widely used AGNEWS. We deliberately fix the number of participating clients to one—i.e., the worst-case scenario without client parallelism—to isolate the pure algorithmic advantage of FwdLLM+ over FwdLLM. The convergence trajectories on these tasks are depicted in Figs. 10 (BERT series) and 11 (OPT-1.3B). We compare against a broad set of baselines spanning BP-based PEFT (e.g., LoRA [38]) and forward-only zeroth-order optimization (e.g., MeZO [62]).

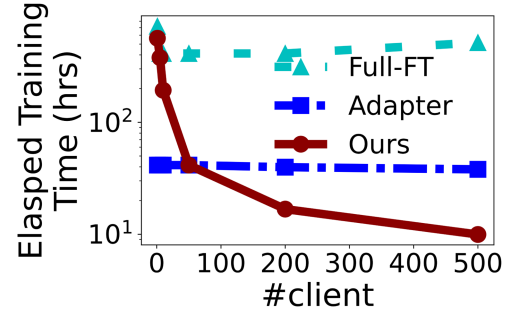


Fig. 12. System performance with different #clients.

Across both model families, FwdLLM+ consistently delivers superior accuracy: on the BERT-series tasks (Fig. 10) it achieves on average 5.7% higher final accuracy than FwdLLM, 14% higher than Adapter, 6.6% higher than MeZO and 16.4% higher than LoRA, while on OPT-1.3B (Fig. 11) the corresponding gains rise to 19.5%, 9%, 0.7% and 7.8% higher, respectively. In addition, FwdLLM+ even surpasses full fine-tuning on the challenging COPA, CB, RTE and WIC/WSC tasks.

Our system exhibits enhanced scalability in convergence speed as the number of available training devices increases: Fig. 12 demonstrates that our system adeptly leverages a growing number of clients to bolster convergence performance. With the client count increasing from 1 to 500, our system’s convergence duration notably reduces from 563.65 to 9.96 minutes. On the other hand, vanilla FedLLM methods, which rely on backward passes, fail to showcase a similar trend in scalability. For these approaches, the convergence duration plateaus once the client count exceeds 5. This behavior echoes the findings in Section II-B, underscoring that our system outperforms backward pass-based FedLLM methods in scalability as more clients are added.

B. Convergence Performance on LLaMA-7B

Discriminative task: We further assess our system using a billion-sized model, LLaMA-7B, on the AGNEWS dataset and the Pixel 7 Pro. To accommodate the model on these devices, we employed the quantization technique GPTQ [73], which is prevalently utilized with LLMs to minimize parameter redundancy. During the fine-tuning process, we adopted low-precision data formats like INT8/INT4 for the native LLaMa weights, while retaining FP32 for the trainable update parameters. Given that backpropagation-based training approaches are not feasible for such sizable models on mobile platforms, we conducted experiments using both centralized training (leveraging 1x NVIDIA A100) and federated learning (on smartphones).

With results shown in Table V, we make following key observations. (1) For the first time, our system enables federated learning of billion-sized LLMs like LLaMA on COTS mobile devices. Combined with INT4 quantization, it takes only 0.19 hours for our system to achieve the target accuracy running on mobile CPU, which is comparable to a centralized training with one NVIDIA A100 GPU. This is mainly due to its ability to scale out its speed on a thousand devices simultaneously. If

TABLE V

OUR SYSTEM COMBINED WITH INT4-BASED QUANTIZATION IS THE ONLY FEASIBLE APPROACH IN FEDERATED LEARNING OF LLAMA-7B. DATASET: AGNEWS; ACC.: ACCURACY (%). CENTRALIZED TRAINING AND FEDERATED LEARNING ARE CONDUCTED ON NVIDIA A100 AND PIXEL 7 PRO, RESPECTIVELY.*: LLAMA CURRENTLY IS NOT SUPPORTED BY MOBILE NPU, THEREFORE WE EMULATE ITS SPEED BASED ON THE SPEEDUP OF OTHER BERT-LIKE MODELS.

Methods	Mem. (GB)	Centralized Training (A100)			Federated Learning		
		Acc.	Round	Time	Acc.	Round	Time
BP, FP16	39.2	89.7	500	0.1 hrs	N/A due to memory inefficiency on Pixel 7 Pro (8GB)		
BP, INT8	32.4	88.6	500	0.06 hrs			
BP, INT4	28.5	87.8	500	0.04 hrs			
Ours, FP16	15.6	87.0	240	1.5 hrs			
Ours, INT8	7.9	86.9	260	0.8 hrs			
Ours (CPU), INT4	4.0	85.8	130	0.25 hrs	85.8	130	0.19 hrs
Ours (NPU*), INT4							0.07 hrs

Instruction input :
Context:
 Bethencourt took the title of King of the Canary Islands, as vassal to Henry III of Castile. In 1418, Jean's nephew Maciot de Bethencourt sold the rights to the islands to Enrique Pérez de Guzmán, 2nd Count de Niebla.
Question:
 Who sold the rights?
Answer:
Llama-7B-original: Jean de Bethencourt sold the rights to the islands to Enrique Pérez de Guzmán, 2nd Count de Niebla.
Llama-7B-tuned(backward): Maciot de Bethencourt
Llama-7B-tuned(forward): Jean's nephew Maciot de Bethencourt
Ground Ture: Maciot de Bethencourt

Fig. 13. A case showing how our system guides the LLaMA to follow human instructions.

on-device training can be accelerated by NPU, our system is even much faster than the centralized training. (2) Our system shows strong compatibility with quantization strategies. Quantizing LLaMA weights to INT8 and INT4 effectively reduces the training-time memory footprint from 15.6 GBs to 7.9 GBs and 4.0 GBs. Yet, the accuracy only degrades by less than 1.2%. This shows the great compatibility of our system with existing model compression algorithms.

Generative (instruction-following) task: We also evaluate our system on the generative task SQUAD. As shown in the Fig. 13, our system could generate similar results as the centralized training. Statistically, our system could reach 83% f1-score after 312 rounds instruct tuning, which is 55.2% higher than zero-shot performance. Even compared with centralized BP-based training, our system is only 1.9% lower, which do not affect the generative quality. Note that our system conducts experiments on the federated setting with only forward pass, which is private and fast as we depicted in Table V.

C. System Cost

We analyze the resource cost during FwdLLM+, including the peak memory footprint, total energy and network (both uplink and downlink) expenditure on all participant devices. The experiments are performed with RoBERTa-large and AGNEWS on Pixel 7 Pro. The results are depicted in Fig. 14. Conceptually, FwdLLM uploads PEFT parameters (e.g., LoRA adapters) each round, whereas FwdLLM+ follows the full-parameter low-rank protocol summarized in Methodology (III A): the downlink is dominated by the explicit model-sized update tensor ΔX^t

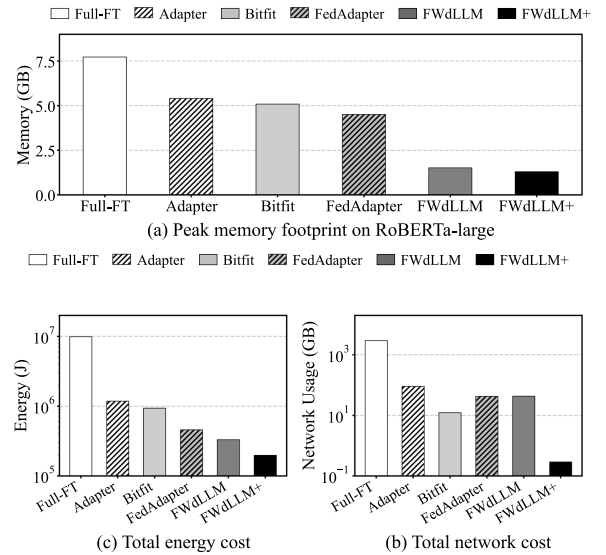


Fig. 14. Resource cost of our system and baselines.

(same order as FedLLM), with only negligible seed overhead for defining the current perturbation directions, while the uplink consists only of low-dimensional scalar coefficients $\{c_{i,\ell}^t\}$.

a) Memory footprint: As shown in Fig. 14, our system achieves up to a 93% reduction in memory usage compared to FT and a 91.3% decrease relative to the robust PEFT benchmarks. Notably, only 169 MB of memory is required for a single round of our system, rendering it highly suitable for mobile devices. This efficiency stems from our BP-free design: clients only execute forward passes and do not materialize backpropagation activations or gradients, so peak memory is dominated by model weights plus a small number of perturbation buffers (cf. Section III-A). Moreover, due to our system's sole reliance on inference functions, cutting-edge inference-centric memory optimization techniques [89], [90], [91] can be effortlessly incorporated to further reduce memory consumption.

b) Energy and network: Compared to Full-FT, our system is able to save 96.7% energy consumption and 98.6% network cost across devices. Our system is still network-efficient compared to competitive baselines with PEFT enhancements, e.g., $2.1\times$ less network traffic compared to Adapter approach. The only expectation is BitFit because it only transmits bias of each parameter in the network, i.e., only 0.1% of the total parameters.

In the aspects of energy consumption, our system consumes $2.6\times$ on average less energy than all the PEFT baselines. The main reason behind that is our system only need to compute forward pass, which is $3\text{-}5\times$ quicker than a backward pass [28], [92] and could be accelerated by NPU processors.

VI. CONVERGENCE ANALYSIS

This section provides a lightweight convergence discussion for FwdLLM+. Our algorithm is a federated instantiation of the low-rank zeroth-order optimization scheme studied in LOZO [68]. Rather than reproducing the full LOZO proof, we isolate the only new ingredient introduced by federation: averaging K independent client estimators yields a $1/K$ variance reduction, which directly improves convergence stability.

A. Problem Formulation

We consider the standard federated objective

$$\min_{X \in \mathbb{R}^d} f(X) := \frac{1}{K} \sum_{i=1}^K f_i(X),$$

$$f_i(X) := \mathbb{E}_{\xi_i \sim \mathcal{D}_i} [F_i(X; \xi_i)]. \quad (5)$$

Where X denotes the full model parameters, K is the number of clients, and client i samples mini-batches ξ_i from its local data distribution $\mathcal{D}_i$.

B. Low-Rank ZO Update and Subspace View

At round t , client i draws a low-rank perturbation direction $Z_i^t = U_i^t (V_i^t)^\top$ (generated from the broadcast seed) and computes a two-point directional derivative

$$c_i^t := \frac{F_i(X^t + \mu Z_i^t; \xi_i^t) - F_i(X^t - \mu Z_i^t; \xi_i^t)}{2\mu}. \quad (6)$$

It then forms a matrix-valued estimator $\hat{g}_i^t := c_i^t \cdot Z_i^t$. The server aggregates

$$\hat{g}_{\text{Fed}}^t := \frac{1}{K} \sum_{i=1}^K \hat{g}_i^t, \quad X^{t+1} := X^t - \eta \hat{g}_{\text{Fed}}^t. \quad (7)$$

Following LOZO [68], one may interpret the update as a zeroth-order subspace optimization procedure: under the “lazy sampling” strategy (holding a projection factor fixed within a short cycle), the low-rank perturbations restrict the search directions to a low-dimensional subspace, and the update is equivalent to applying ZO-SGD on a projected surrogate objective. In what follows, we only use LOZO to justify that $\hat{g}_i^t$ is a valid ZO gradient estimator with an “effective dimension” dependence controlled by the rank.

C. Assumptions

We adopt standard smoothness and stochasticity assumptions used in non-convex ZO/Sgd analysis [31], [63], together with the subspace sampling condition from LOZO [68].

Assumption 1 (Smoothness): The objective f is differentiable and L -smooth: $\|\nabla f(X) - \nabla f(Y)\| \leq L\|X - Y\|$.

Assumption 2 (ZO estimator properties): Let $f_{i,\mu}$ denote the (subspace) smoothed objective associated with the two-point estimator on client i , and let $f_\mu := \frac{1}{K} \sum_{i=1}^K f_{i,\mu}$. Conditioned on X^t , each client’s estimator is unbiased for its smoothed gradient and has bounded variance:

$$\mathbb{E}[\hat{g}_i^t | X^t] = \nabla f_{i,\mu}(X^t), \quad \mathbb{E}\|\hat{g}_i^t - \nabla f_{i,\mu}(X^t)\|^2 \leq \sigma^2. \quad (8)$$

Assumption 3 (Independence across clients): Conditioned on X^t , the mini-batches $\{\xi_i^t\}_{i=1}^K$ and low-rank directions $\{Z_i^t\}_{i=1}^K$ are independent across clients.

D. Federated Variance Reduction

The key benefit of federation in FwdLLM+ is that the server averages K independent matrix estimators.

Lemma 1 (Variance reduction): Under Assumptions 2–3,

$$\mathbb{E}\|\hat{g}_{\text{Fed}}^t - \nabla f_\mu(X^t)\|^2 = \frac{1}{K^2} \sum_{i=1}^K \mathbb{E}\|\hat{g}_i^t - \nabla f_{i,\mu}(X^t)\|^2 \leq \frac{\sigma^2}{K}. \quad (9)$$

The proof of Lemma 1 and the full convergence derivation are provided in the Supplementary Appendix H.

E. Convergence Result

Applying standard non-convex SGD analysis to the smoothed objective f_μ , with the variance term replaced by σ^2/K from Lemma 1, yields the usual stationarity guarantee. In particular, with a suitable step size schedule (capped by $1/L$), the iterates satisfy

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\|\nabla f_\mu(X^t)\|^2 \leq \mathcal{O}\left(\frac{1}{\sqrt{KT}}\right) + \mathcal{O}(\mu^2), \quad (10)$$

where the hidden constants include the effective-dimension dependence induced by the low-rank subspace (see LOZO [68]), and $\mathcal{O}(\mu^2)$ is the irreducible smoothing bias. This highlights why increasing the number of clients improves convergence stability: the stochasticity of the ZO estimator is reduced linearly in K .

VII. RELATED WORK

FedLLM: With the recent rise of transformers and their variants [33], [93], [94], [95], [96], LLMs have achieved great success in various domains, including CV, NLP, etc. While its success is largely attributed to the pre-training paradigm, the privacy issue of LLMs has been a major concern, e.g., extraction attacks [97], [98], [99]. FedLLM is a promising direction to relieve the privacy tension of LLMs [100]. To tackle the tight resource constraint on devices, most recent efforts resort to parameter-efficient fine-tuning (PEFT) techniques such as Adapter and LoRA [38], [39], [74], [101]. For instance, FedAdapter [6] addresses the adapter configuration problem for FedLLM. However, in Section II we show that PEFT-based FedLLM only mitigates the network bottleneck, but cannot significantly optimize the memory or speed of on-device training. FwdLLM is built atop PEFT but significantly reduces the

device-side overhead and accelerates convergence. A concurrent work, MeZO [62], presents a more mathematical study on how forward gradients can be leveraged to enable more resource-efficient LLM finetuning. Inspired by LOZO [68], which builds upon MeZO and investigates low-rank perturbations for model optimization, we promote FwdLLM+, adapting and further extending the low-rank perturbation technique to the FedLLM setting through our own novel mechanisms.

Backpropagation-free training: Backpropagation is the most widely adopted but not the only way to train neural networks, such as zeroth-order optimization proposed in early 80s [102]. As models grow larger, researchers are realizing that backpropagation becomes a burden to the DNN training pipeline. Thus, a few new paradigms for backpropagation-free training are proposed [28], [103], [104], [105]. This work is built atop forward gradient for being mobile friendly. However, prior literature [28], [29], [59] of forward gradient still use toy models (MLP or small CNNs). BBTv2 [105] applies gradient-free methods to optimize the LLM prompts in a central cloud. BAFFLE [106] and FedZeN [107] are two concurrent works to FwdLLM, which combine backpropagation-free training with FL. However, these approaches are primarily evaluated on small or medium-size models; when scaling to LLMs, their zeroth-order estimators require a large number of random perturbations across many parameter directions, resulting in substantial on-device computation and communication overhead. Another concurrent work FedKSeed [108] aims to enable full-parameter zeroth-order optimization of FedLLMs; it reduces the network cost by updating a scalar gradient accumulator instead of forward gradients vectors. However, it requires hundreds of seconds to decode the gradients from the scalar accumulator and excessive local training steps for each client.

FL Optimizations: There have been tremendous efforts in making cross-device FL more efficient, including communication efficiency [109], [110], model compression/quantization [111], [112], client/data sampling [45], [46], [48], [113], [114], [115], [116], [117], and on-device training speedup [18], [37], [118]. However, their benefits are modest compared to the gap between the LLMs like LLaMA and the resource constraint of mobile devices. FwdLLM+ innovates the FL protocol by abandoning backpropagation and therefore brings much more significant improvements.

VIII. CONCLUSION

In this study, we introduce FwdLLM+, the pioneering federated learning framework for LLMs that dispenses with backpropagation by exploiting forward gradients, an adaptive perturbation generator, and a discriminative sampler, thereby achieving faster convergence and better scalability than contemporary FedLLM methods; building upon these advances, FwdLLM+ further mitigates precision, perturbation, and efficiency issues through full-parameter fine-tuning with low-rank-aware perturbations, attaining superior convergence, memory efficiency, and scalability for billion-parameter LLMs on mobile devices while maintaining comparable communication costs.

As a limitation, our empirical validation currently covers models up to 7B parameters and common pre-training corpora.

The effectiveness and convergence characteristics on substantially larger models or under alternative pre-training objectives and data distributions remain to be verified.

Looking ahead, future work will extend the FwdLLM+ framework to multimodal large language models (MLLMs). While FwdLLM+ mitigates the memory wall by eliminating backpropagation, challenges remain for large multimodal inputs. The multiple forward passes required for gradient estimation may become a computational bottleneck on mobile devices when processing high-resolution media. Furthermore, our low-rank perturbation strategy was designed for LLMs; its applicability to the heterogeneous components of MLLMs (e.g., vision encoders) is an open question. Future research should therefore focus on developing adaptive perturbation strategies for MLLMs and validating our optimization approach on larger-scale multimodal tasks.

ACKNOWLEDGMENT

This work was completed while Dongqi Cai was a visiting researcher at the University of Cambridge.

REFERENCES

- [1] M. Xu, F. Qian, Q. Mei, K. Huang, and X. Liu, "DeepType: On-device deep learning for input personalization service with minimal privacy concern," *Proc. ACM on Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 2, no. 4, pp. 1–26, 2018.
- [2] L. Jin et al., "EMSAAssist: An end-to-end mobile voice assistant at the edge for emergency medical services," in *Proc. 21st Annu. Int. Conf. Mobile Syst., Appl. Serv.*, 2023, pp. 275–288.
- [3] W. Yin, M. Xu, Y. Li, and X. Liu, "LLM as a system service on mobile devices," 2024, *arXiv:2403.11805*.
- [4] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "Mobile{BERT}: A compact task-agnostic {BERT} for resource-limited devices," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 2158–2170. [Online]. Available: <https://aclanthology.org/2020.acl-main.195>
- [5] J. Yuan et al., "Mobile foundation model as firmware," in *Proc. 30th Annu. Int. Conf. Mobile Comput. Netw.*, 2024, pp. 279–295.
- [6] D. Cai, Y. Wu, S. Wang, F. X. Lin, and M. Xu, "Efficient federated learning for modern NLP," in *Proc. 29th Annu. Int. Conf. Mobile Comput. Netw.*, 2023, pp. 1–16.
- [7] J. Zhang et al., "Towards building the {FederatedGPT}: Federated instruction tuning," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2024, pp. 6915–6919, doi: [10.1109/ICASSP48485.2024.10447454](https://doi.org/10.1109/ICASSP48485.2024.10447454).
- [8] D. Cai, S. Wang, Y. Wu, F. X. Lin, and M. Xu, "Federated few-shot learning for mobile NLP," in *Proc. 29th Annu. Int. Conf. Mobile Comput. Netw.*, 2023, pp. 1–17.
- [9] H. Zhao, W. Du, F. Li, P. Li, and G. Liu, "FedPrompt: Communication-efficient and privacy-preserving prompt tuning in federated learning," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2023, pp. 1–5.
- [10] Y. Tian, Y. Wan, L. Lyu, D. Yao, H. Jin, and L. Sun, "FedBERT: When federated learning meets pre-training," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 4, pp. 1–26, Aug. 2022, doi: [10.1145/3510033](https://doi.org/10.1145/3510033).
- [11] B. Wang et al., "Can public large language models help private cross-device federated learning?," in *Proc. Findings Assoc. Comput. Linguistics*, 2024, pp. 934–949. [Online]. Available: <https://aclanthology.org/2024.findings-naacl.59>
- [12] P. Goyal et al., "Accurate, large minibatch SGD: Training ImageNet in 1 hour," 2017, *arXiv:1706.02677*.
- [13] E. Hoffer, I. Hubara, and D. Soudry, "Train longer, generalize better: Closing the generalization gap in large batch training of neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 1731–1741. [Online]. Available: <https://papers.nips.cc/paper/2017/hash/a5e0ff62be0b08456fc7f1e88812af3d-Abstract.html>
- [14] Y. You, Y. Wang, H. Zhang, Z. Zhang, J. Demmel, and C.-J. Hsieh, "The limit of the batch size," 2020, *arXiv:2006.08517*.

- [15] P. Zeng et al., “The cap principle for LLM serving,” 2024, *arXiv:2405.11299*.
- [16] M. Xu et al., “A survey of resource-efficient LLM and multimodal foundation models,” 2024, *arXiv:2401.08092*.
- [17] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” 2023, *arXiv:2302.13971*.
- [18] Q. Wang et al., “Melon: Breaking the memory wall for resource-efficient on-device machine learning,” in *Proc. 20th Annu. Int. Conf. Mobile Syst., Appl. Serv.*, 2022, pp. 450–463.
- [19] H. Cai, C. Gan, L. Zhu, and S. Han, “Tiny{TL}: Reduce memory, not parameters for efficient on-device learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 11285–11297. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/81f7acabd411274fcf65ce2070ed568a-Abstract.html>
- [20] S. Lee and S. Nirjon, “Fast and scalable in-memory deep multitask learning via neural weight virtualization,” in *Proc. 18th Int. Conf. Mobile Syst., Appl., Serv.*, 2020, pp. 175–190.
- [21] X. Peng et al., “Capuchin: Tensor-based GPU memory management for deep learning,” in *Proc. 25th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2020, pp. 891–905.
- [22] Android, “Android: Low memory killer daemon,” 2022. [Online]. Available: <https://source.android.com/docs/core/perf/lmkd>
- [23] Google, “Select TensorFlow operators,” Google AI Edge, 2026. Accessed: Mar. 24, 2026. [Online]. Available: https://ai.google.dev/edge/litert/conversion/tensorflow/ops_select
- [24] “Federated learning: Collaborative machine learning without centralized training data,” 2017. [Online]. Available: <https://ai.googleblog.com/2017/04/federated-learning-collaborative.html>
- [25] P. J. Werbos, “Backpropagation through time: What it does and how to do it,” *Proc. IEEE*, vol. 78, no. 10, pp. 1550–1560, Oct. 1990.
- [26] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [27] M. Xu, D. Cai, Y. Wu, X. Li, and S. Wang, “[FwdLLM]: Efficient federated finetuning of large language models with perturbed inferences,” in *Proc. USENIX Annu. Tech. Conf.*, 2024, pp. 579–596.
- [28] A. G. Baydin, B. A. Pearlmutter, D. Syme, F. Wood, and P. Torr, “Gradients without backpropagation,” 2022, *arXiv:2202.08587*.
- [29] M. Ren, S. Kornblith, R. Liao, and G. Hinton, “Scaling forward gradient with local losses,” in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–31. [Online]. Available: <https://openreview.net/forum?id=XjpBP1JM15>
- [30] C. M. Stein, “Estimation of the mean of a multivariate normal distribution,” *Ann. Statist. Statist.*, vol. 9, no. 6, pp. 1135–1151, 1981.
- [31] S. Liu et al., “A primer on zeroth-order optimization in signal processing and machine learning: Principles, recent advances, and applications,” *IEEE Signal Process. Mag.*, vol. 37, no. 5, pp. 43–54, Sep. 2020.
- [32] A. D. Flaxman, A. T. Kalai, and H. B. McMahan, “Online convex optimization in the bandit setting: Gradient descent without a gradient,” in *Proc. 16th Annu. ACM-SIAM Symp. Discrete Algorithms*, 2005, pp. 385–394.
- [33] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “[BERT]: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, Vol. 1 (Long Short Papers), 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423>
- [34] Y. Liu et al., “Roberta: A robustly optimized bert pretraining approach,” 2019, *arXiv:1907.11692*.
- [35] S. Zhang et al., “OPT: Open pre-trained transformer language models,” 2022, *arXiv:2205.01068*.
- [36] C. Chen, X. Feng, J. Zhou, J. Yin, and X. Zheng, “Federated large language model: A position paper,” 2023, *arXiv:2307.08925*.
- [37] D. Xu et al., “Mandheling: Mixed-precision on-device DNN training with DSP offloading,” in *Proc. 28th Annu. Int. Conf. Mobile Comput. Netw.*, 2022, pp. 214–227.
- [38] E. J. Hu et al., “LoRA: Low-rank adaptation of large language models,” in *Int. Conf. Learn. Representations*, 2021, pp. 1–26. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [39] J. Pfeiffer et al., “[A]dapter[H]ub: A framework for adapting transformers,” in *Proc. Conf. Empirical Methods Natural Lang. Process., System Demonstrations*, 2020, pp. 46–54. [Online]. Available: <https://aclanthology.org/2020.emnlp-demos.7>
- [40] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, 2023.
- [41] N. Lebeck, A. Krishnamurthy, H. M. Levy, and I. Zhang, “End the senseless killing: Improving memory management for mobile operating systems,” in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 873–887.
- [42] C. Li, J. Bao, and H. Wang, “Optimizing low memory killers for mobile devices using reinforcement learning,” in *Proc. 13th Int. Wireless Commun. Mobile Comput. Conf.*, 2017, pp. 2169–2174.
- [43] A.I.ETH Zurich, Switzerland, “AI-benchmark ranking,” 2024. Accessed: Mar. 24, 2026. [Online]. Available: <https://ai-benchmark.com/ranking.html>
- [44] J. S. Jeong et al., “Band: Coordinated multi-DNN inference on heterogeneous mobile processors,” in *Proc. 20th Annu. Int. Conf. Mobile Syst., Appl. Serv.*, 2022, pp. 235–247.
- [45] A. Li, J. Sun, P. Li, Y. Pu, H. Li, and Y. Chen, “Hermes: An efficient federated learning framework for heterogeneous mobile clients,” in *Proc. 27th Annu. Int. Conf. Mobile Comput. Netw.*, 2021, pp. 420–437.
- [46] C. Li, X. Zeng, M. Zhang, and Z. Cao, “PyramidFL: A fine-grained client selection framework for efficient federated learning,” in *Proc. 28th Annu. Int. Conf. Mobile Comput. Netw.*, 2022, pp. 158–171.
- [47] J. Shin, Y. Li, Y. Liu, and S.-J. Lee, “FedBalancer: Data and pace control for efficient federated learning on heterogeneous clients,” in *Proc. 20th Annu. Int. Conf. Mobile Syst., Appl. Serv.*, 2022, pp. 436–449.
- [48] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, “Oort: Efficient federated learning via guided participant selection,” in *Proc. 15th {USENIX} Symp. Operating Syst. Des. Implementation*, 2021, pp. 19–35. [Online]. Available: <https://www.usenix.org/conference/osdi21/presentation/lai>
- [49] B. Yuchen Lin et al., “FedNLP: Benchmarking federated learning methods for natural language processing tasks,” in *Proc. Findings Assoc. Comput. Linguistics*, 2022, pp. 157–175.
- [50] Y. Kim, J. Kim, W.-L. Mok, J.-H. Park, and S. Lee, “Client-customized adaptation for parameter-efficient federated learning,” in *Proc. Findings Assoc. Comput. Linguistics*, 2023, pp. 1159–1172.
- [51] K. Luo, X. Li, Y. Lan, and M. Gao, “GradMA: A gradient-memory-based accelerated federated learning with alleviated catastrophic forgetting,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3708–3717.
- [52] F. Lai et al., “FedScale: Benchmarking model and system performance of federated learning at scale,” in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 11814–11827.
- [53] S. Horvath, S. Laskaridis, M. Almeida, I. Leontiadis, S. Venieris, and N. Lane, “FjORD: Fair and accurate federated learning under heterogeneous targets with ordered dropout,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 12876–12889.
- [54] J. Zhang, E. Chen, C. Liu, and C. G. Brinton, “DPZV: Elevating the tradeoff between privacy and utility in zeroth-order vertical federated learning,” 2025, *arXiv:2502.20565*.
- [55] E. Bao et al., “Unlocking the power of differentially private zeroth-order optimization for fine-tuning {LLMs},” in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 1569–1588.
- [56] K. Bonawitz et al., “Practical secure aggregation for privacy-preserving machine learning,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191, doi: [10.1145/3133956.3133982](https://doi.org/10.1145/3133956.3133982).
- [57] P. Correia et al., “Federated learning: An approach with hybrid homomorphic encryption,” 2025, *arXiv:2509.03427*.
- [58] X. Tang, A. Panda, M. Nasr, S. Mahloulifar, and P. Mittal, “Private fine-tuning of large language models with zeroth-order optimization,” *Trans. Mach. Learn. Res.*, vol. 2025, pp. 1–27, 2025. [Online]. Available: <https://collaborate.princeton.edu/en/publications/private-fine-tuning-of-large-language-models-with-zeroth-order-op>
- [59] S. Park, D. Shin, J. Chung, and N. Lee, “FedFwd: Federated learning without backpropagation,” in *Proc. Federated Learn. Anal. Practice, Algorithms, Syst., Appl., Opportunities*, 2023, pp. 1–8.
- [60] Z. Zhang, B. Liu, and J. Shao, “Fine-tuning happens in tiny subspaces: Exploring intrinsic task-specific subspaces of pre-trained language models,” in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (Vol. 1, Long Papers)*, 2023, pp. 1701–1713. [Online]. Available: <https://aclanthology.org/2023.acl-long.95>
- [61] J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar, and Y. Tian, “[G]a[L]ore: Memory-efficient {LLM} training by gradient low-rank projection,” in *Proc. 41st Int. Conf. Mach. Learn.*, in ser. Proceedings of Machine Learning Research, 2024, vol. 235, pp. 61\121–61\143. [Online]. Available: <https://proceedings.mlr.press/v235/zhao24s.html>

- [62] S. Malladi et al., "Fine-tuning language models with just forward passes," in *Proc. Adv. Neural Inf. Process. Syst.*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023, vol. 36, pp. 53038–53075. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/a627810151be4d13f907ac898ff7e948-Paper-Conference.pdf
- [63] S. Ghadimi and G. Lan, "Accelerated gradient methods for nonconvex nonlinear and stochastic programming," *Math. Program.*, vol. 156, no. 1, pp. 59–99, 2016.
- [64] Y. Zhang et al., "Revisiting zeroth-order optimization for memory-efficient {LLM} fine-tuning: A benchmark," in *Proc. 41st Int. Conf. Mach. Learn.*, in ser. Proceedings of Machine Learning Research, 2024, vol. 235, pp. 59\,173–59\,190. [Online]. Available: <https://proceedings.mlr.press/v235/zhang24ad.html>
- [65] X. Chen et al., "ZO-AdaMM: Zeroth-order adaptive momentum method for black-box optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., 2019, vol. 32, pp. 7204–7215. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/576d026223582a390cd323bef4bad026-Abstract.html>
- [66] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by error propagation," Inst. Cognitive Sci., Univ. California, San Diego, La Jolla, CA, USA, ICS Tech. Rep. 8506, Sep. 1985.
- [67] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge, U.K.: Cambridge Univ. Press, 2020.
- [68] Y. Chen, Y. Zhang, L. Cao, K. Yuan, and Z. Wen, "Enhancing zeroth-order fine-tuning for language models with low-rank structures," in *Proc. Int. Conf. Learn. Representations*, 2025, pp. 1–27. [Online]. Available: <https://openreview.net/forum?id=9BiVepgmWW>
- [69] PyTorch Contributors, "functorch," PyTorch documentation. Accessed: Mar. 24, 2026. [Online]. Available: <https://docs.pytorch.org/functorch/stable/>
- [70] AutoGPTQ Contributors, "AutoGPTQ," GitHub repository, 2023. Accessed: Mar. 24, 2026. [Online]. Available: <https://github.com/AutoGPTQ/AutoGPTQ>
- [71] Google, "LiteRT (formerly TensorFlow Lite)," Google AI Edge, 2024. Accessed: Mar. 24, 2026. [Online]. Available: <https://ai.google.dev/edge/litert>
- [72] ggml-org, "llama.cpp," GitHub repository, 2023. Accessed: Mar. 24, 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [73] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "{OPTQ}: Accurate quantization for generative pre-trained transformers," in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–16. [Online]. Available: <https://openreview.net/forum?id=tcbBPnfwxS>
- [74] E. Ben Zaken, Y. Goldberg, and S. Ravfogel, "{B}it{F}it: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Vol. 2, Short Papers)*, 2022, pp. 1–9. [Online]. Available: <https://aclanthology.org/2022.acl-short.1>
- [75] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Natural Lang. Process., System Demonstrations*, 2020, pp. 38–45. [Online]. Available: <https://aclanthology.org/2020.emnlp-demos.6>
- [76] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, vol. 28, pp. 649–657. [Online]. Available: <https://proceedings.neurips.cc/paper/2015/hash/250cf8b51c773f3f8dc8b4be867a9a02-Abstract.html>
- [77] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "{SQuAD}: 100,000+ questions for machine comprehension of text," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2016, pp. 2383–2392. [Online]. Available: <https://aclanthology.org/D16-1264>
- [78] A. Wang et al., "Superglue: A stickier benchmark for general-purpose language understanding systems," in *Proc. Adv. Neural Inf. Process. Syst.*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., 2019, vol. 32, pp. 3266–3280. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/4496bf24afe7fab6f046bf4923da8de6-Abstract.html>
- [79] R. Socher et al., "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2013, pp. 1631–1642.
- [80] I. Dagan, O. Glickman, and B. Magnini, "The pascal recognising textual entailment challenge," in *Proc. Mach. Learn. Challenges Workshop*, 2005, pp. 177–190.
- [81] H. J. Levesque, E. Davis, and L. Morgenstern, "The winograd schema challenge," in *Proc. 13th Int. Conf. Princ. Knowl. Representation Reasoning*, 2012. [Online]. Available: <https://aaai.org/papers/59-4492-the-winograd-schema-challenge/>
- [82] M.-C. De Marneffe, M. Simons, and J. Tonhauser, "The commitment-bank: Investigating projection in naturally occurring discourse," in *Proc. Sinn und Bedeutung*, 2019, vol. 23, no. 2, pp. 107–124.
- [83] M. T. Pilehvar and J. Camacho-Collados, "{W}i{C}: The word-in-context dataset for evaluating context-sensitive meaning representations," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, Vol. 1 (Long Short Papers), 2019, pp. 1267–1273. [Online]. Available: <https://aclanthology.org/N19-1128>
- [84] M. Roemmele, C. A. Bejan, and A. S. Gordon, "Choice of plausible alternatives: An evaluation of commonsense causal reasoning," in *Proc. AAAI Spring Symp.: Log. Formalizations Commonsense Reason.*, 2011, pp. 90–95.
- [85] NVIDIA, "Jetson TX2 Module," NVIDIA Developer, 2017. Accessed: Mar. 24, 2026. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-tx2>
- [86] A. Paszke et al., "{PyTorch}: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32, pp. 8026–8037. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- [87] N. Housley et al., "Parameter-efficient transfer learning for NLP," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2790–2799.
- [88] H. B. McMahan, E. Moore, D. Ramage, and B. A. y Arcas, "Federated learning of deep networks using model averaging," 2016, *arXiv:1602.05629*.
- [89] X. Liu, H. Peng, N. Zheng, Y. Yang, H. Hu, and Y. Yuan, "EfficientViT: Memory efficient vision transformer with cascaded group attention," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14420–14430.
- [90] X. Liang, J. Li, L. Wu, Z. Cao, and M. Zhang, "Dynamic and efficient inference for text generation via bert family," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguist. (Vol. 1: Long Papers)*, 2023, pp. 2883–2897.
- [91] S. S. Vatsavai, V. S. P. Karempudi, and I. Thakkar, "An optical XNOR-bitcount based accelerator for efficient inference of binary neural networks," in *Proc. 24th Int. Symp. Qual. Electron. Des.*, 2023, pp. 1–8.
- [92] É. Blayo, M. Bocquet, E. Cosme, and L. F. Cugliandolo, *Adv. Data Assimilation Geosciences: Lecture Notes the LES Houches Sch. Phys. Special Issue, June 2012*. Oxford, U.K.: OUP Oxford, 2014.
- [93] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30. [Online]. Available: <https://academic.oup.com/book/6390>
- [94] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.
- [95] L. Hou, Z. Huang, L. Shang, X. Jiang, X. Chen, and Q. Liu, "DynaBERT: Dynamic BERT with adaptive width and depth," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 9782–9793.
- [96] W. Liu, P. Zhou, Z. Zhao, Z. Wang, H. Deng, and Q. Ju, "FastBERT: A self-distilling BERT with adaptive inference time," 2020, *arXiv:2004.02178*.
- [97] J. Deng et al., "TAG: Gradient attack on transformer-based language models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 6035–6044. [Online]. Available: <https://aclanthology.org/2020.acl-main.537/>
- [98] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32, pp. 3600–3610. [Online]. Available: <https://aclanthology.org/2021.findings-emnlp.305/>
- [99] M. Balunovic, D. Dimitrov, N. Jovanović, and M. Vechev, "LAMP: Extracting text from gradients with language model priors," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 7641–7654.
- [100] W. Zhuang, C. Chen, and L. Lyu, "When foundation model meets federated learning: Motivations, challenges, and future directions," 2023, *arXiv:2306.15546*.
- [101] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, and G. Neubig, "Towards a unified view of parameter-efficient transfer learning," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 1–14.
- [102] A. G. Barto and M. I. Jordan, "Gradient following without back-propagation in layered networks," in *Proc. 1st Int. Conf. Neural Nets*, San Diego, CA, USA, vol. 2, 2022, pp. 1–15. [Online]. Available: <https://iclr.cc/virtual/2022/spotlight/6525>
- [103] G. Hinton, "The forward-forward algorithm: Some preliminary investigations," 2022, *arXiv:2212.13345*.

- [104] W.-D. K. Ma, J. Lewis, and W. B. Kleijn, "The HSIC bottleneck: Deep learning without back-propagation," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 04, pp. 5085–5092.
- [105] T. Sun, Z. He, H. Qian, Y. Zhou, X.-J. Huang, and X. Qiu, "BBTV2: Towards a gradient-free future with large language models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2022, pp. 3916–3930.
- [106] H. Feng, T. Pang, C. Du, W. Chen, S. Yan, and M. Lin, "Does federated learning really need backpropagation?" 2023, *arXiv:2301.12195*.
- [107] H. Feng, T. Pang, C. Du, W. Chen, S. Yan, and M. Lin, "BAFFLE: A baseline of backpropagation-free federated learning," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 89–109.
- [108] A. Maritan, S. Dey, and L. Schenato, "Fed{ZeN}: Quadratic convergence in zeroth-order federated learning via incremental hessian estimation," in *Proc. Eur. Control Conf.*, 2024, pp. 2320–2327, doi: [10.23919/ECC64448.2024.10590749](https://doi.org/10.23919/ECC64448.2024.10590749).
- [109] K. Bonawitz et al., "Towards federated learning at scale: System design," in *Proc. Mach. Learn. Syst.*, 2019, vol. 1, pp. 374–388.
- [110] C. Yang et al., "Characterizing impacts of heterogeneity in federated learning upon large-scale smartphone data," in *Proc. Web Conf.*, 2021, pp. 935–946.
- [111] J. Wu, W. Huang, J. Huang, and T. Zhang, "Error compensated quantized SGD and its applications to large-scale distributed optimization," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5325–5333.
- [112] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signSGD: Compressed optimisation for non-convex problems," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 560–569.
- [113] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *Proc. IEEE Int. Conf. Commun.*, 2019, pp. 1–7.
- [114] J. Xu and H. Wang, "Client selection and bandwidth allocation in wireless federated learning networks: A long-term perspective," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 1188–1200, Feb. 2021.
- [115] S. Wang, M. Lee, S. Hosseinalipour, R. Morabito, M. Chiang, and C. G. Brinton, "Device sampling for heterogeneous federated learning: Theory, algorithms, and implementation," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [116] Y. Zhao and X. Gong, "Quality-aware distributed computation and user selection for cost-effective federated learning," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2021, pp. 1–6.
- [117] A. Li, L. Zhang, J. Tan, Y. Qin, J. Wang, and X.-Y. Li, "Sample-level data selection for federated learning," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [118] J. Yuan et al., "Towards energy-efficient federated learning via int8-based training on mobile DSPs," in *Proc. ACM Web Conf.*, 2024, pp. 2786–2794.



Chen Peng is currently working toward the master's degree in computer science with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. Her research interests include federated learning and resource-efficient AI systems.



Mengwei Xu received the BS and PhD degrees from Peking University, Beijing, China. He is currently an associate professor with the Beijing University of Posts and Telecommunications, Beijing. His research interests include mobile and edge computing.



Zhenyan Lu is currently working toward the PhD degree in computer science with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. Her research interests include mobile computing and resource-efficient AI systems.



Wei Liu received the bachelor's and master's degrees from the Beijing University of Posts and Telecommunications. He is currently a senior algorithm engineer specializing in large models with Xiaomi Group.



Shangguang Wang (Senior Member, IEEE) received the PhD degree with the Beijing University of Posts and Telecommunications in 2011. He is currently a professor with the School of Computer Science and Engineering, Beijing University of Posts and Telecommunications, China. He has authored or coauthored more than 150 papers. His research interests include service computing, mobile edge computing, and satellite computing. He is currently serving as chair of IEEE Technical Committee on Services Computing (2022–2023), and vice-chair of IEEE Technical Committee on Cloud Computing since 2020. He also served as general chairs or program chairs of more than 10 IEEE conferences. He is a fellow of the IET. For further information, please visit: <http://www.sgguangwang.com>.



Nicholas D. Lane is currently a professor with the Department of Computer Science and Technology, University Cambridge, U.K., where he leads the Machine Learning Systems Lab (CaMLSys). Alongside his academic role, he is also a program director with the Samsung AI Center in Cambridge where his teams study on-device and distributed forms of machine learning.



Qibo Sun received the PhD degree in communication and electronic system from the Beijing University of Posts and Telecommunication. He is currently an associate professor with the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, China. His research interests include services computing, satellite computing, and space-ground integration network. He is a member of the China Computer Federation and Chinese Association for Artificial Intelligence.



Dongqi Cai (Member, IEEE) is currently working toward the PhD degree with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. He is a visiting PhD student with the University of Cambridge, U.K. His research interests include mobile computing and federated learning.